nano-rare GT Framework — Architecture Decision Record 001 Status: Accepted Date: 2025-05-14

Context

We need a computational framework that matches nano-rare monogenic diseases with viable gene therapy surrogates. The architecture must be: 1. Reproducible (bioinformaticians demand version-controllable, local-first compute) 2. Modular (each scoring dimension independently testable) 3. Fast to iterate (SQLite for v0, PostgreSQL for production) 4. CLI-first (scriptable, CI-friendly, no heavy UI dependencies) *What is a CLI?|* it's a Command Line Interface -> its a way to control a program by typing commands into a terminal, instead of clicking buttons in app -> they're fast and good for working with code, servers and tools. For examples: git status npm install python script.py

Decision

1. **Python 3.10+** — dominant language in bioinformatics; excellent async support; mature testing ecosystem. for core framework
2. **Pydantic v2** — runtime validation, JSON serialization, type safety without boilerplate. for typed, validated data models
3. **SQLite v0.1, PostgreSQL v0.2** — SQLite is zero-config, single-file, portable. PostgreSQL adds concurrency and pgvector for embeddings when we need scale. later when scale/concurrency/vector search matters
4. **Typer for CLI** — modern, type-hint-driven, auto-generates help with commands like `nanogt match`, `nanogt init`, `nanogt status`.
5. **Jinja2 for reports** — Markdown templates version-controllable, diffable, human-readable. for generated markdown reports
6. **requests-cache** — all external APIs cached with 7-day TTL by default. Prevents pipeline stalls from rate limits. so external API calls are cached and reproducible-ish
7. **pytest + mypy + black + ruff** — standard Python quality stack. Pre-commit hooks enforce style on every commit. for code quality

important product idea is v0.1 should be installable and usable with someing like - pip install nanogt -, no Docker or hosted service required. ROGDI is the 'golden path' test case - so before a scoring module counts as done, it needs to work against the ROGDI fixture

Consequences

Positive

- Single `pip install nanogt` gets a working system with no Docker or external DB needed.
- Each scoring module (`disease.py`, `homology.py`, …) can be unit-tested in isolation.
- SQLite DB is a single portable file; perfect for sharing reproducible analyses.

Negative

- SQLite does not support concurrent writes well; batch mode must serialize or use WAL.
- No native vector search in SQLite v0.1; semantic similarity will use in-memory cosine until pgvector in v0.2.
- Local-first means no hosted API in v0.1 (FastAPI layer deferred to v0.2).

ROGDI as Primary Validation Target

The first validation dataset is ROGDI / Kohlschütter-Tönz syndrome: - Gene size: ~1044 bp CDS → trivially fits AAV - Well-defined protein: Q9P2T1, 348 aa, IMPDH domain - Clear cell types: hippocampal neurons + ameloblasts - No existing GT trials → perfect nano-rare candidate

All modules must pass the ROGDI test fixture before they are considered complete.